

Nauka HTMLa od podstaw

Damian Żłobicki · Accessibility First

DAMIAN ŻŁOBICKI

- Andragog
- Trener biznesu
- **Lider Sieci Dostępności Cyfrowej**
- Cyfrowy Coach
- Prezenter i konferansjer
- **CEO Włącz Wizję,**
- Founder Accessibility First i Alholics

ACCESSIBILITY FIRST



Plan na dziś

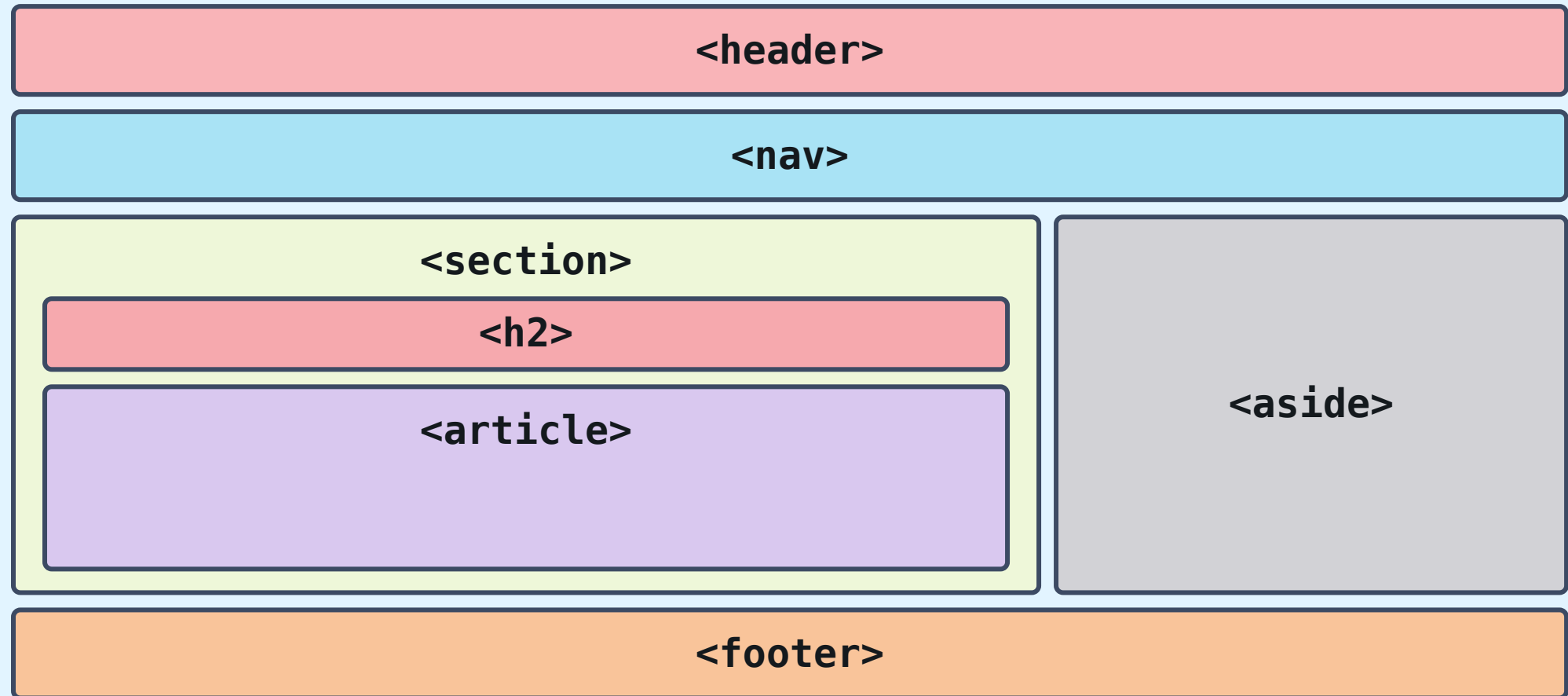
- **Część 1. Teoria:** od historii HTML do semantyki
- **Część 2. Warsztat:** 7 tematów w praktyce
- Pytania i odpowiedzi

Część 1

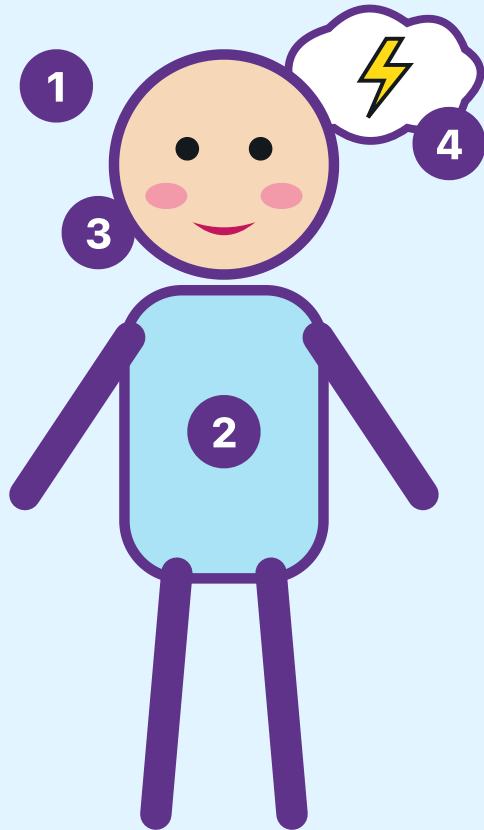
Teoria

Od historii HTML do semantyki

Tak wygląda strona w HTML



HTML jak człowiek



1. Głowa = `<head>`

myśli i szczegóły, których nie widać: tytuł, język, opis strony

2. Ciało = `<body>`

to, co widać, i to, z czym wchodzimy w interakcję

3. CSS = makijaż

wygląd: kolory, czcionki, układ

4. JavaScript = humory

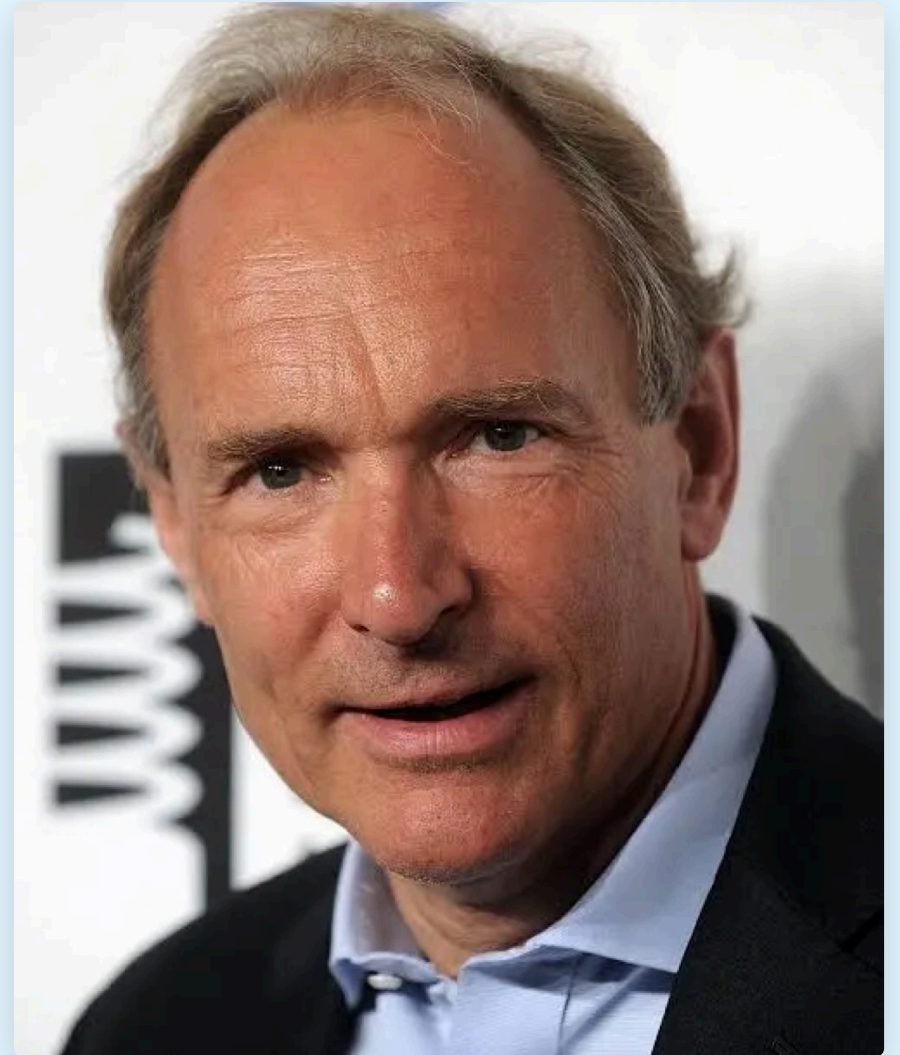
dynamiczne zmiany: reakcje na kliknięcia, rozwijanie, podmiana treści

Po co powstał HTML?

- CERN, 1989: naukowcy nie mogą łatwo dzielić się wiedzą
- Pomysł: dokumenty połączone linkami
- Prosty zapis struktury, czytelny dla każdego komputera
- Od początku: **znaczenie treści, nie wygląd**

Tim Berners-Lee, W3 i W3C

- 1989: propozycja World Wide Web
- 1990-1991: pierwszy HTML, przeglądarka, serwer i strona
- 1993: CERN udostępnia WWW wszystkim, bez opłat
- 1994: powstaje W3C, konsorcjum standardów sieci



1991: od czego zaczął się HTML

title a p h1-h6 ul li dl address

Najedź na znacznik albo przejdź Tabem.

Zero znaczników wyglądu.

Standardy HTML

- 1995: HTML 2.0, pierwszy standard (IETF, RFC 1866)
- 1997: HTML 3.2 i HTML 4.0 (W3C)
- 1999: HTML 4.01 · 2000: XHTML 1.0
- 2004: powstaje WHATWG (Apple, Mozilla, Opera)
- 2014: HTML5, rekomendacja W3C (28 października)

WCAG: po co powstało?

- 1997: W3C uruchamia WAI (Web Accessibility Initiative)
- 1999: WCAG 1.0 (5 maja)
- 2008: WCAG 2.0, zasady niezależne od technologii
- 2018: WCAG 2.1 · 2023: WCAG 2.2 (5 października)
- WCAG 2.2: 9 nowych kryteriów, usunięte 4.1.1 Parsowanie

WCAG na świecie

- USA: Sekcja 508, od 2018 odwołuje się do WCAG 2.0 AA
- USA: ADA Tytuł II (2024), władze stanowe i lokalne, WCAG 2.1 AA
- UE: dyrektywa 2016/2102 i norma EN 301 549
- UE: Europejski Akt o Dostępności, od 28.06.2025
- Polska: ustawy z 4.04.2019 i z 26.04.2024

ARIA: po co powstała?

- Lata 2000.: strony stają się aplikacjami
- Div udający przycisk: czytnik przeczyta tekst, ale nie poda roli ani stanu
- ARIA dodaje role, stany i właściwości
- ARIA 1.0: 2014 · 1.1: 2017 · 1.2: 2023

CSS i JavaScript

- CSS pomaga: kontrast, widoczny fokus, powiększanie, mniej animacji
- CSS szkodzi: inna kolejność wizualna niż w kodzie, ukryta treść
- JS pomaga: dynamiczne treści, zarządzanie fokusem, klawiatura
- JS szkodzi: klikalny div bez klawiatury, zmiana bez komunikatu

Stan na dziś

- HTML: Living Standard (WHATWG), jedna wersja, stale aktualizowana
- 2019: W3C i WHATWG uzgadniają jedną specyfikację HTML
- WCAG 2.2: najnowsza wersja W3C
- Prawo w PL i UE odsyła dziś do WCAG 2.1 AA
- WCAG 3.0: szkic roboczy, **nie** standard
- ARIA 1.2 obowiązuje, kolejna wersja w pracach

HTML i ARIA dziś

- Większość tego, co wymagało ARIA, HTML ma natywnie
- „ARIA in HTML” (W3C): kiedy wolno dodać rolę, a kiedy nie
- Pierwsza zasada ARIA: **masz natywny element, użyj go**
- ARIA zmienia to, co słyszy czytnik, nie zachowanie elementu

HTML dziś: najpopularniejsze znaczniki

div

28,7% elementów

Co czwarty element w sieci nic nie znaczy dla czytnika ekranu.

a

12,6% elementów

Jest od 1991 r., bez niego nie byłoby sieci. Dziś często udaje przycisk.

span

11,2% elementów

Tak samo „pusty” semantycznie jak **div**, tylko w linii tekstu.

li

7,7% elementów

Czytnik poda liczbę elementów listy, ale tylko gdy to prawdziwa **ul** albo **ol**.

img

3,3% elementów

Bez **alt** czytnik może przeczytać nazwę pliku.

button

na 73% stron

Za darmo daje rolę, fokus i obsługę klawiaturą. W 2021 r. był na 65,5% stron.

font · center

po 4,5% stron

Przestarzałe od lat, a wciąż są mniej więcej na co 22. stronie.

mediana

32 różne elementy

Typowa strona ma 594 elementy, ale tylko 32 rodzaje. Wystarcza niewielki zestaw znaczników.

Źródło: HTTP Archive, Web Almanac 2024, rozdział Markup, strony mobilne. „% elementów” to udział we wszystkich elementach, nie odsetek stron.

Czym jest semantyczny HTML?

- Element mówi, **czym jest** treść, a nie jak wygląda
- Przeglądarka daje za darmo: rolę, nazwę, fokus, klawiaturę
- **No ARIA is better than bad ARIA**
- Zła ARIA kłamie czytnikowi. Brak ARIA nie wprowadza w błąd.

Budowa strony i typowe błędy

- Szkielet: `header`, `nav`, `main`, `footer` i atrybut `lang`
- Brak tekstu alternatywnego: 53,1% stron
- Pola bez etykiet: 51% · puste linki: 46,3%
- Puste przyciski: 30,6%

Część 2

Warsztat

Instrukcja na slajdzie, potem pokaz na żywo

Narzędzia: klawiatura, czytnik ekranu, DevTools

1. Nagłówki i landmarky

- Dobra praktyka: jeden `h1`, poziomy bez przeskoków (nie wymóg WCAG)
- Nagłówek to tytuł sekcji, nie duża czcionka
- Landmarki: `header`, `nav`, `main`, `footer`
- **Sprawdź:** NVDA: H, 1-6, D, Insert+F7 · VoiceOver: VO+U

2. Obrazki

- Każdy `img` ma atrybut `alt`
- Informacyjny: alt mówi, **co przekazuje**
- Dekoracyjny: `alt=""`
- W linku: alt mówi, **dokąd prowadzi**
- SVG: `role="img"` + `aria-label` · wykres: krótki alt + pełny opis obok

3. Link czy przycisk?

- Zmienia się adres: **link** (`a href`)
- Coś dzieje się na stronie: **przycisk** (`button`)
- **Sprawdź:** Enter działa na obu, Spacja tylko na przycisku
- Tekst linku zrozumiały w kontekście (2.4.4, AA)
- `button` w formularzu domyślnie wysyła: `type="button"`

4. Div i span

- Nie znaczą nic, i to jest w porządku
- Dobrze do układu i stylowania
- Problem: gdy udają przycisk, link, nagłówek albo listę
- **Sprawdź:** czy dojdiesz Tabem? Co ogłasza czytnik?
- Jeśli musisz: `role="button"` + `tabindex="0"` + Enter i Spacja w JS

Div jak button: test 1:1

`<button>`

Zapisz

Zapisano 0 razy

1 linia HTML, 0 linii JS do klawiatury

`<div role="button">`

Zapisz

Zapisano 0 razy

`tabindex` + Enter + Spacja w JS

☐ Div tylko z `onclick` (wersja z recenzji)

Sprawdź: Tab do każdego, potem Enter i Spacja.

Ile kosztuje div

✓ Wystarczy to

```
<button type="button">Zapisz</button>
```

Div, żeby działał tak samo

```
<div role="button" tabindex="0" id="zapisz">Zapisz</div>
<script>
  const el = document.getElementById("zapisz");
  el.addEventListener("click", zapisz);
  el.addEventListener("keydown", (e) => {
    if (e.key === "Enter") { e.preventDefault(); el.click(); }
    if (e.key === " ") e.preventDefault(); // bez przewijania
  });
  el.addEventListener("keyup", (e) => {
    if (e.key === " ") el.click();          // Spacja po puszczeniu
  });
</script>
```

Czego div i tak nie dostanie

- `disabled`: wyłączenie z fokusu i kliknięć
- `type="submit"`: wysłanie formularza, także Enterem w polu
- `name` i `value` przy wysyłaniu formularza
- Każdą z tych rzeczy dopisujesz i testujesz sam

5. Listy

- `ul`: kolejność bez znaczenia · `ol`: kolejność ma znaczenie
- Menu nawigacji to też lista
- Czytnik ogłasza listę i liczbę pozycji (brzmienie zależy od czytnika)
- Safari: `list-style: none` może zgubić listę, obejście
`role="list"`

6. Tabele

- Tabela na dane, nigdy do układu strony
- `caption`: tytuł tabeli
- `th` i `scope`: nagłówki kolumn i wierszy
- Tylko 19% tabel ma poprawne znaczniki danych

7. Formularze

- Każde pole ma `label` (`for` i `id`)
- Placeholder to podpowiedź, nie etykieta
- Grupy pól: `fieldset` i `legend`
- Błąd powiązany z polem · `autocomplete` przy danych osobowych
- Pole wymagane: `required` + słowo „wymagane”, nie sam kolor ani gwiazdka

h1 · ING: mówimy prostym językiem

lista · linki · obrazek

button div

tabela · Zespół ING

h3 · Zarejestruj się (*pusto*)

formularz

podsumowanie błędów

pole z błędem

podpowiedź pod polem

Błędne pole nie robi się czerwone

Zmienne `--blad` i `--ok` nie istnieją, więc styl błędu znika bez ostrzeżenia.

Poprawka: zdefiniuj je w `:root`.

2. Brak `main` i skiplinka

Czytnik nie ma punktu „treść główna”, klawiatura nie przeskoczy na początek treści. **Poprawka:** `<a href="#tresc">` i `<main id="tresc">`.

3. Pusty nagłówek

Pod `h3` nie ma treści. W liście nagłówków to sekcja-widmo. **Poprawka:** usuń go.

4. `div` udaje przycisk

Tab dochodzi, czytnik mówi „przycisk”, ale Enter i Spacja nic nie robią.

Poprawka: `<button>`.

5. Nazwa na zwykłym `div`

`aria-labelledby` na elemencie bez roli jest niedozwolone i bywa pomijane. **Poprawka:** usuń atrybut.

6. Widać, a czytnik nie zna

`aria-hidden` na widocznej podpowiedzi. W trybie czytania jej nie ma.

Poprawka: usuń `aria-hidden`.

Polecane źródła

- a11yfirst.pl: materiały i baza wiedzy
- [CodePen](https://codepen.io): ćwiczenia na żywo
- [W3Schools](https://www.w3schools.com): szybkie przykłady składni
- [MDN](https://developer.mozilla.org) · [W3C WAI Tutorials](https://www.w3.org/WAI/tutorials)

ACCESSIBILITY FIRST

Q&A

Damian Żłobicki

13.05.26



włączwizję

ACCESSIBILITY FIRST

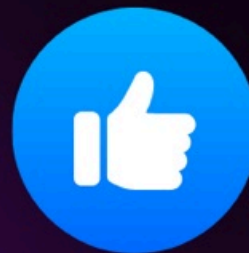
a11yfirst.pl

Dziękujemy za uwagę

DZIĘKUJĘ ZA UWAGĘ



Obserwuj mnie



damian@włączwizje.pl

ACCESSIBILITY FIRST



włączwizję